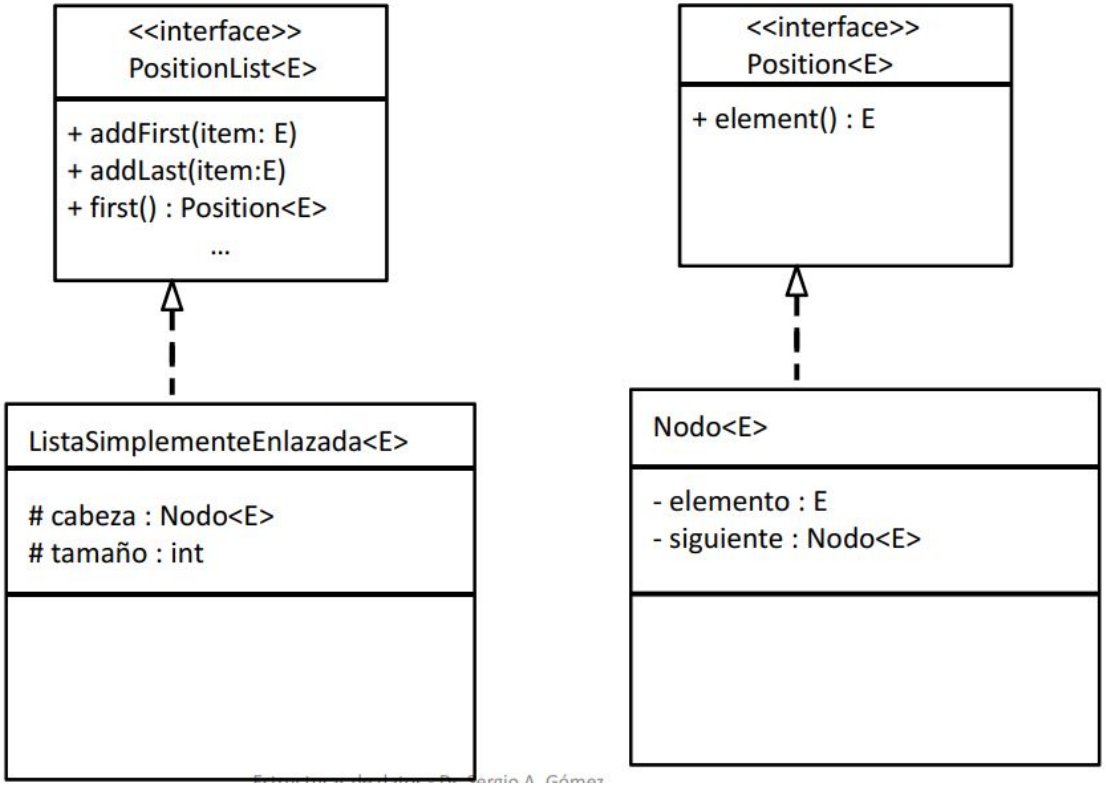
Estructuras de Datos

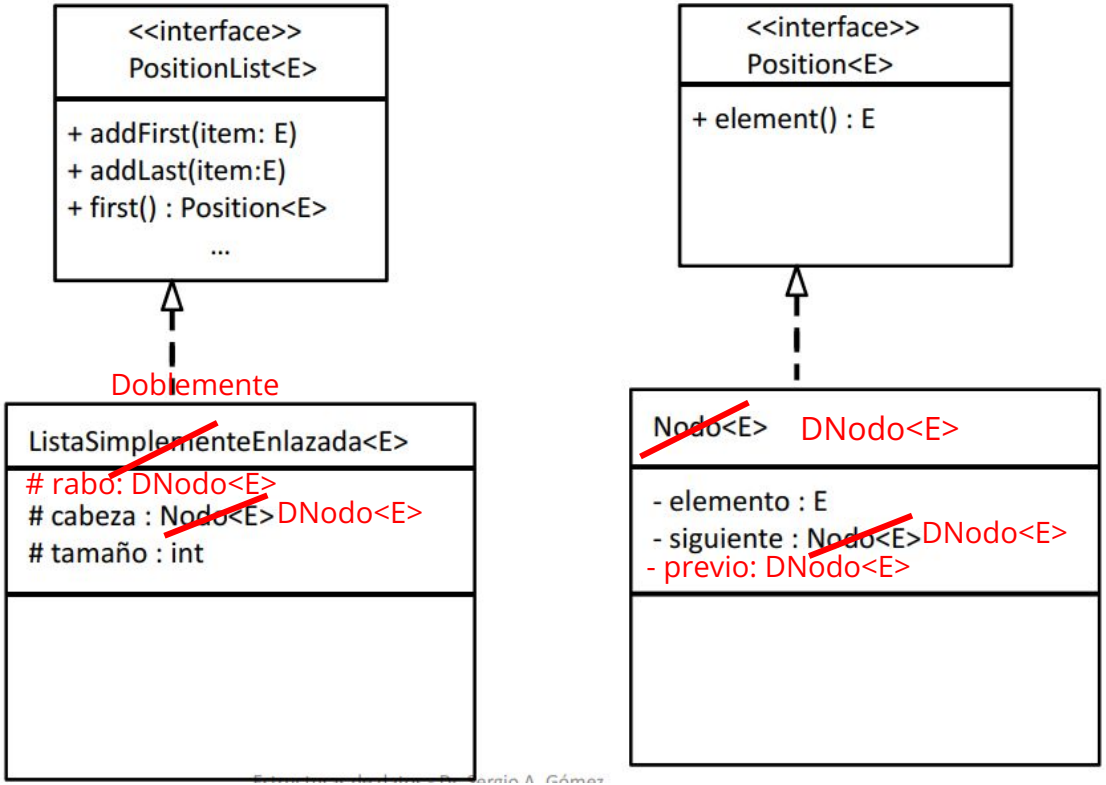
2026
Primer Cuatrimestre

REPASO

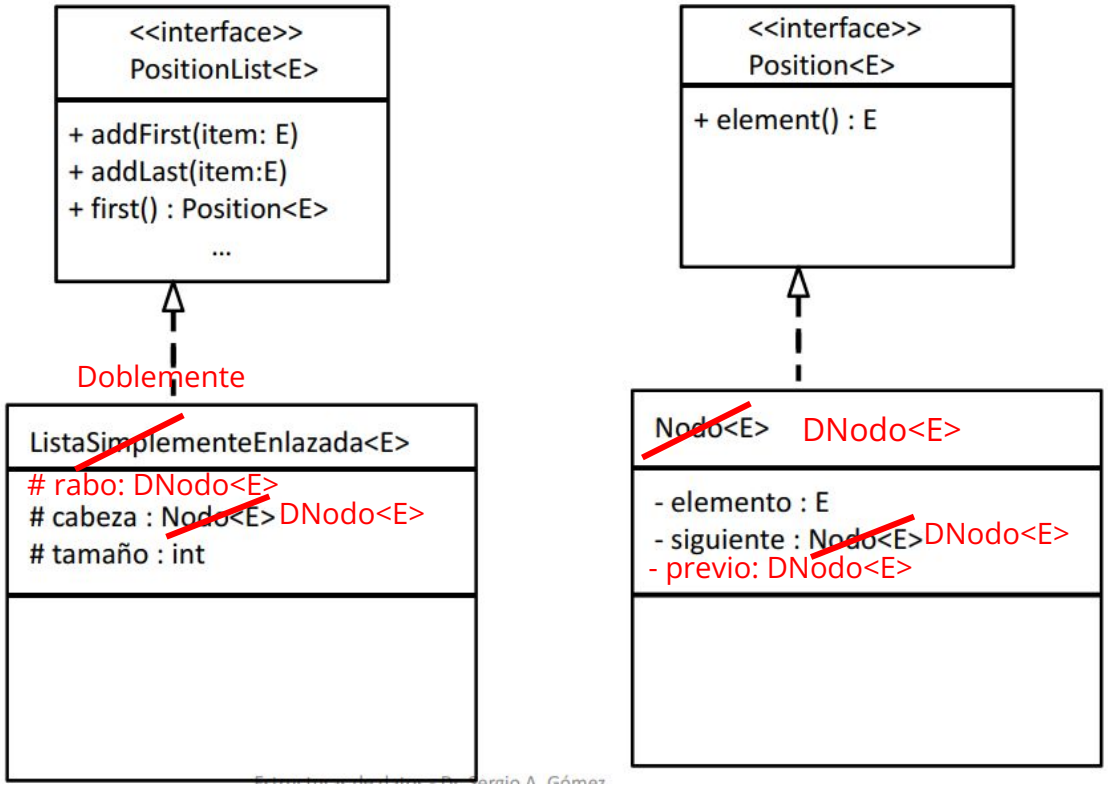
ListaSimplementeEnlazada: Diseño UML



ListaDoblementeEnlazada: Diseño UML



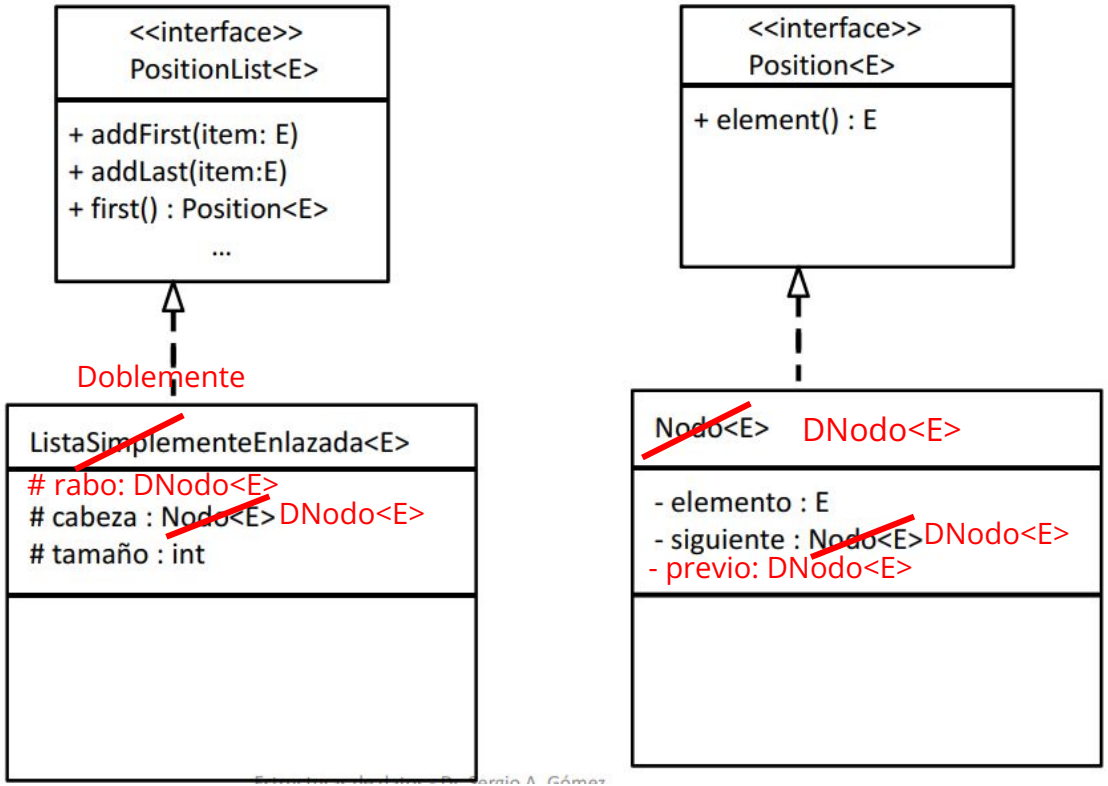
ListaDoblementeEnlazada: Diseño UML




```
public E obtenerPrimerElem (PositionList<E> lista){
   if (lista.size()>=1){
         Node<E> pos = lista.first()
   }
}
```

¿Puedo hacer esto?

ListaDoblementeEnlazada: Diseño UML

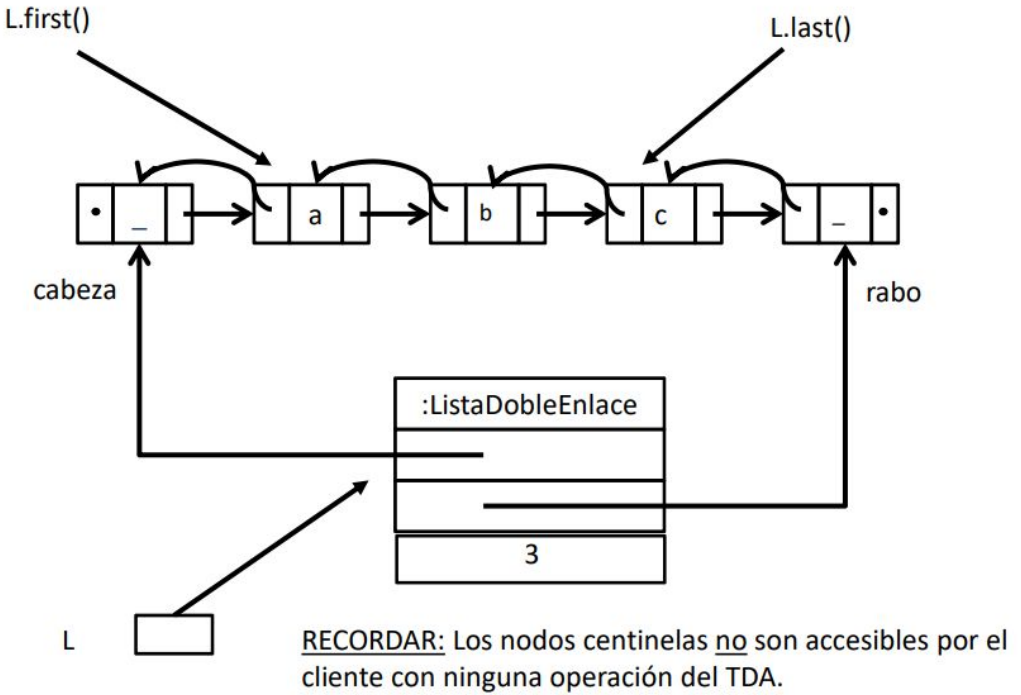



```
public E obtenerPrimerElem (PositionList<E> lista){
   if (lista.size>=1){
         Position<E> pos = lista.first()
   }
}
```

```
public E obtenerSegundoElem (PositionList<E> lista){
   if (lista.size>=2){
         Position<E> pos = lista.first()
         pos = lista.next(pos) // pos = pos.siguiente
   }
}
```

Verificar siempre si tengo que trabajar con el TDA o con la estructura

TDA Lista con DNodo<E> + Position + con Enlace al Final y con Centinelas

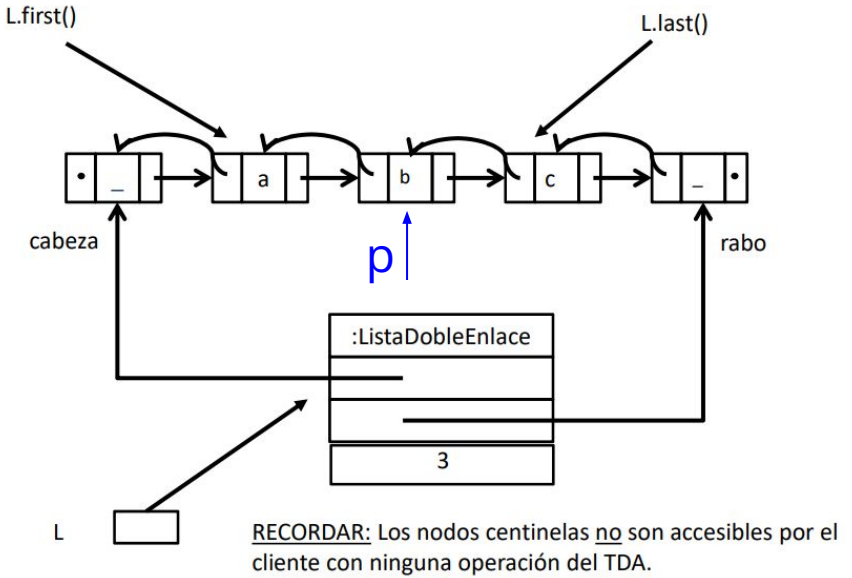


AddAfter

```
public void addAfter(Position<E> p, E element) {



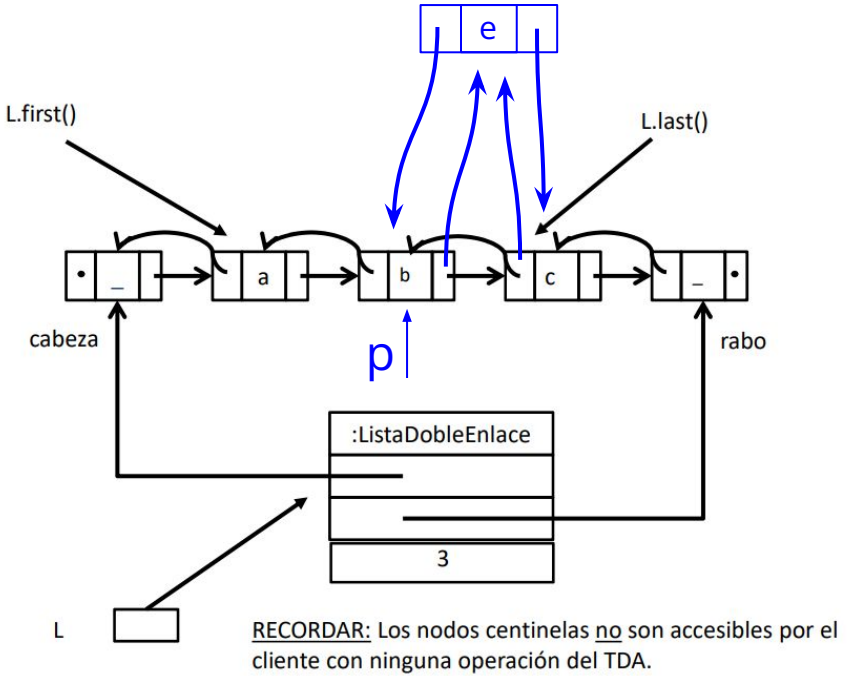
}
```



```
public void addAfter(Position<E> p, E element) {

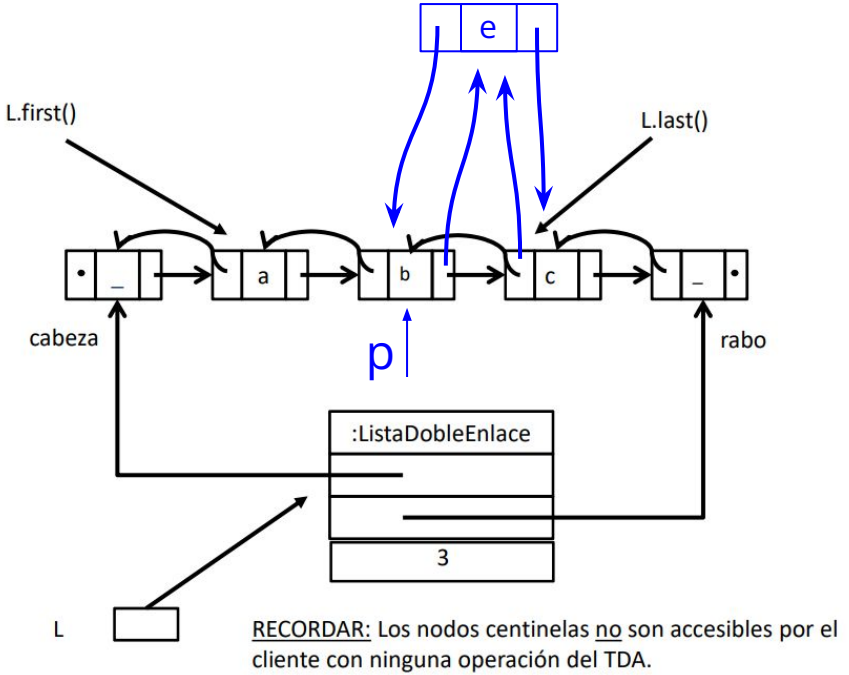



}
```



AddAfter

```
public void addAfter(Position<E> p, E element) {
        DNodo<E> pos = checkPosition(p);
        DNodo<E> nuevo = new DNodo<E>(element);
        nuevo.setNext(pos.getNext());
        nuevo.setPrev(pos);
        nuevo.getNext().setPrev(nuevo);
        pos.setNext(nuevo);
        size++;
}
```



Lista con Nodo<E> Doblemente Enlazado + Posición		
Métodos del TDA	¿Se resuelve inmediato?	¿Salimos a buscar algo?
first()	Si	No
last()	Si	No
prev(p)	Si	No
next(p)	Si	No
set(p, e)	Si	No
addFirst(e)	Si	No
addLast(e)	Si	No
addBefore(p, e)	Si	No
addAfter(p, e)	Si	No
remove(p)	Si	No

Veamos la interfaz del TDA PositionList

https://github.com/ed-dcic-uns/tdas/blob/main/src/main/java/ar/edu/uns/cs/ed/tdas/tdalista/PositionList.java

- Excepciones
- Operaciones adicionales

Listas de Java: interface List

- Java brinda una versión de listas por medio de la interfaz java.util.List<E>
- Las listas de Java son 0-basadas (el primer elemento es el 0 como en los arreglos)
- Implementa una secuencia L=[x_0, x_1, x_2, ..., x_{n-1}] de elementos de tipo genérico E
  - n es la longitud de L
  - La posición de cada elemento es un entero i entre 0 y n-1 tal que la posición de x_0 es 0, la de x_1 es 1, ..., la de x_{n-1} es n-1.
- Hay dos implementaciones de List<E>: ArrayList<E> y LinkedList<E>.

ArrayList

- add(e): Inserta un elemento al final de la lista (append).
- size(): Retorna la cantidad de elementos de la lista L
- isEmpty(): Retorna verdadero si la lista L está vacía y falso en caso contrario
- get(i): Retorna el elemento i-esimo de la lista L; ocurre un error si i<0 o i>size()-1
- set(i,e): Reemplaza con e al elemento i-esimo; ocurre un error si i<0 o i>size()-1
- add(i,e): Agrega un elemento e en posición i; ocurre un error si i<0 o i>size()
- remove(i): elimina el elemento i-esimo de la lista L; ocurre un error si i<0 o i>size()-1
- clear(): Elimina todos los elementos de la lista
- toArray(): retorna un array con los elementos de la lista en el mismo orden
- indexOf(e): índice de la primera aparición de e en la lista
- lastIndexOf(e): índice de la última aparición de e en la lista

AHORA...

Motivación - Estructuras de Datos

Sin importar que TDA sea, ni cómo esté implementado... Si representa una Estructura de Datos entonces debe brindarme acceso a los datos (de alguna manera).

Motivación - Estructuras de Datos

Sin importar que TDA sea, ni cómo esté implementado... Si representa una Estructura de Datos entonces debe brindarme acceso a los datos (de alguna manera).

Y hay más;

Motivación - Estructuras de Datos

Sin importar que TDA sea, ni cómo esté implementado... Si representa una Estructura de Datos entonces debe brindarme acceso a los datos (de alguna manera).

Y hay más; existe una necesidad fundamental en cualquier Estructura de Datos.

Motivación - Estructuras de Datos

Sin importar que TDA sea, ni cómo esté implementado... Si representa una Estructura de Datos entonces debe brindarme acceso a los datos (de alguna manera).

Y hay más; existe una necesidad fundamental en cualquier Estructura de Datos. No importa que tipo o implementación de ED sea, en algún momento (más seguido de lo que pensamos) vamos a tener que realizar la siguiente acción:

Motivación - Estructuras de Datos

Recorrer Todos Los Elementos

Motivación - Estructuras de Datos

Recorrer Todos Los Elementos

```
Position<E> p = lista.first();
Position<E> fin = lista.last();
while(p != fin) {
        E element = p.element();
        p = lista.next(p);
}
E element = p.element();
```

Motivación - Estructuras de Datos

Recorrer Todos Los Elementos

```
Position<E> p = lista.first();
Position<E> fin = lista.last();
while(p != fin) {
        E element = p.element();
        p = lista.next(p);
}
E element = p.element();
```

Para cada elemento de mi lista {
hacer algo con ese elemento
}

Motivación - Estructuras de Datos

Recorrer Todos Los Elementos

```
Position<E> p = lista.first();
Position<E> fin = lista.last();
while(p != fin) {
        E element = p.element();
        p = lista.next(p);
}
E element = p.element();
```

Para cada elemento de mi estructura {
hacer algo con ese elemento
}

Motivación - Estructuras de Datos

Esto ocurre tan seguido, que se han buscado formas de optimizar el proceso. Incluso, algunos lenguajes incluyen palabras reservadas específicas para esta tarea.

ITERADORES

Iteradores

El objetivo de este concepto es poder recorrer los elementos de una ED haciendo una abstracción de cómo se hace dicho recorrido.

Iteradores

método f(Estructura de Datos ED) {

Iteradores

método f(Estructura de Datos ED) {

Iterador i = ED.dameUnIterador(); //No importa qué tipo de estructura es ED

Iteradores

```
método f( Estructura de Datos ED) {
Iterador i = ED.dameUnIterador(); //No importa qué tipo de estructura es ED
mientras (i.tenesElementosParaDarme() ) {
        Elemento e = i.dameUnElemento();
        hagoAlgoCon(e);
}
```

Iteradores

Una misma ED puede ser recorrida de diferentes maneras. Por ejemplo, una lista puede recorrerse en forma creciente o decreciente.

Iteradores

```
método f( Estructura de Datos ED) {
Iterador i = ED.dameUnIterador();
mientras (i.tenesElementosParaDarme() ) {
        Elemento e = i.dameUnElemento();
        hagoAlgoCon(e);
}
```

Iteradores

```
método f( Estructura de Datos ED) {
Iterador i = ED.dameUnIterador();
mientras (i.tenesElementosParaDarme() ) {
        Elemento e = i.dameUnElemento();
        hagoAlgoCon(e);
}
```

El iterador me da la seguridad de que estoy recorriendo todos los elementos. Pero desconozco cuál es el criterio que utiliza para ordenarme los elementos

Iteradores

```
método f( Estructura de Datos ED) {
Iterador i = ED.dameUnIteradorCreciente();
mientras (i.tenesElementosParaDarme() ) {
        Elemento e = i.dameUnElemento();
        hagoAlgoCon(e);
}
```

Iteradores

```
método f( Estructura de Datos ED) {
Iterador i = ED.dameUnIteradorDecreciente();
mientras (i.tenesElementosParaDarme() ) {
        Elemento e = i.dameUnElemento();
        hagoAlgoCon(e);
}
```



Díganme los nombres de las capitales de las provincias argentinas



Ustedes me están diciendo los nombres de las capitales de manera secuencial, cuando la estructura subyacente **NO** es secuencial

Iteradores

```
método f( MapaDeArgentina ED) {
Iterador i = ED.dameUnIteradorDeCapitalesDeProvincia();
mientras (i.tenesElementosParaDarme() ) {
        Elemento e = i.dameUnElemento();
        hagoAlgoCon(e);
}
```

Iteradores

```java
public static void main(String[] args) {
    ArrayList<Integer> numeros = new ArrayList<>();

    numeros.add(1);
    numeros.add(2);
    numeros.add(3);
    numeros.add(4);
    numeros.add(5);

    Iterator<Integer> iterador = numeros.iterator();

    while (iterador.hasNext()) {
        Integer numero = iterador.next();
        System.out.println(numero);
    }
}
```

Requiere:
import java.util.ArrayList;
import java.util.Iterator;

Iteradores

```
    Iterator<Integer> iterador = numeros.iterator();

    while (iterador.hasNext()) {
        Integer numero = iterador.next();
        System.out.println(numero);
    }

    for (Integer numero: numeros) {
        System.out.println(numero);
    }
}
```

Esta última variante se conoce como un **foreach**

Iteradores

Así usamos los iteradores, ¿pero cómo ofrecemos iteradores en nuestras propias estructuras?

Iteradores

Así usamos los iteradores, ¿pero cómo ofrecemos iteradores en nuestras propias estructuras?

Iteradores

Para que nuestra ED sea iterable, el primer paso es implementar la interfaz **Iterable<E>**

Iteradores

Para que nuestra ED sea iterable, el primer paso es implementar la interfaz **Iterable<E>**

```
public interface Iterable<E>
{
    public Iterator<E> iterator();
}
```

Iteradores

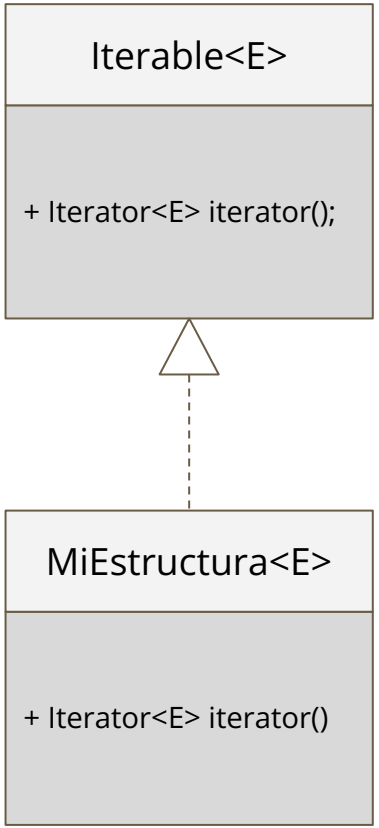


Iteradores



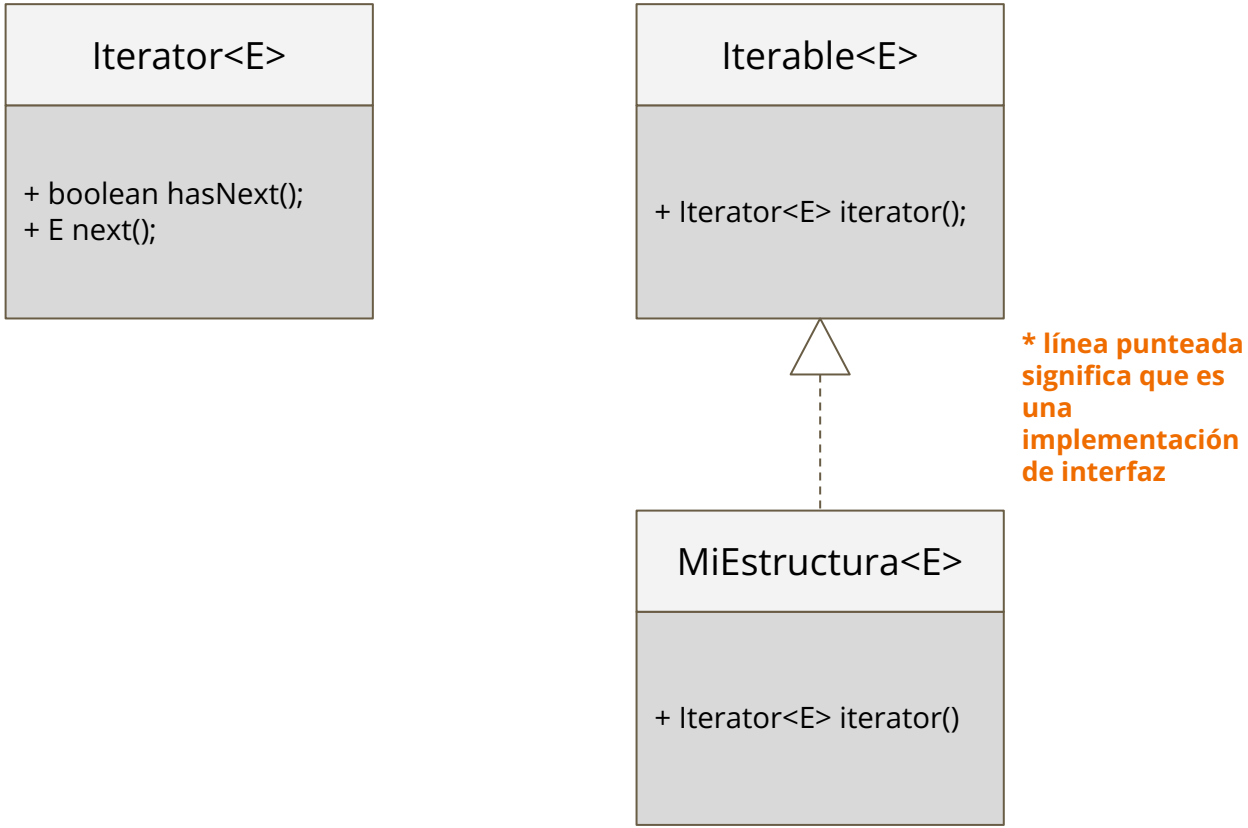


Iteradores

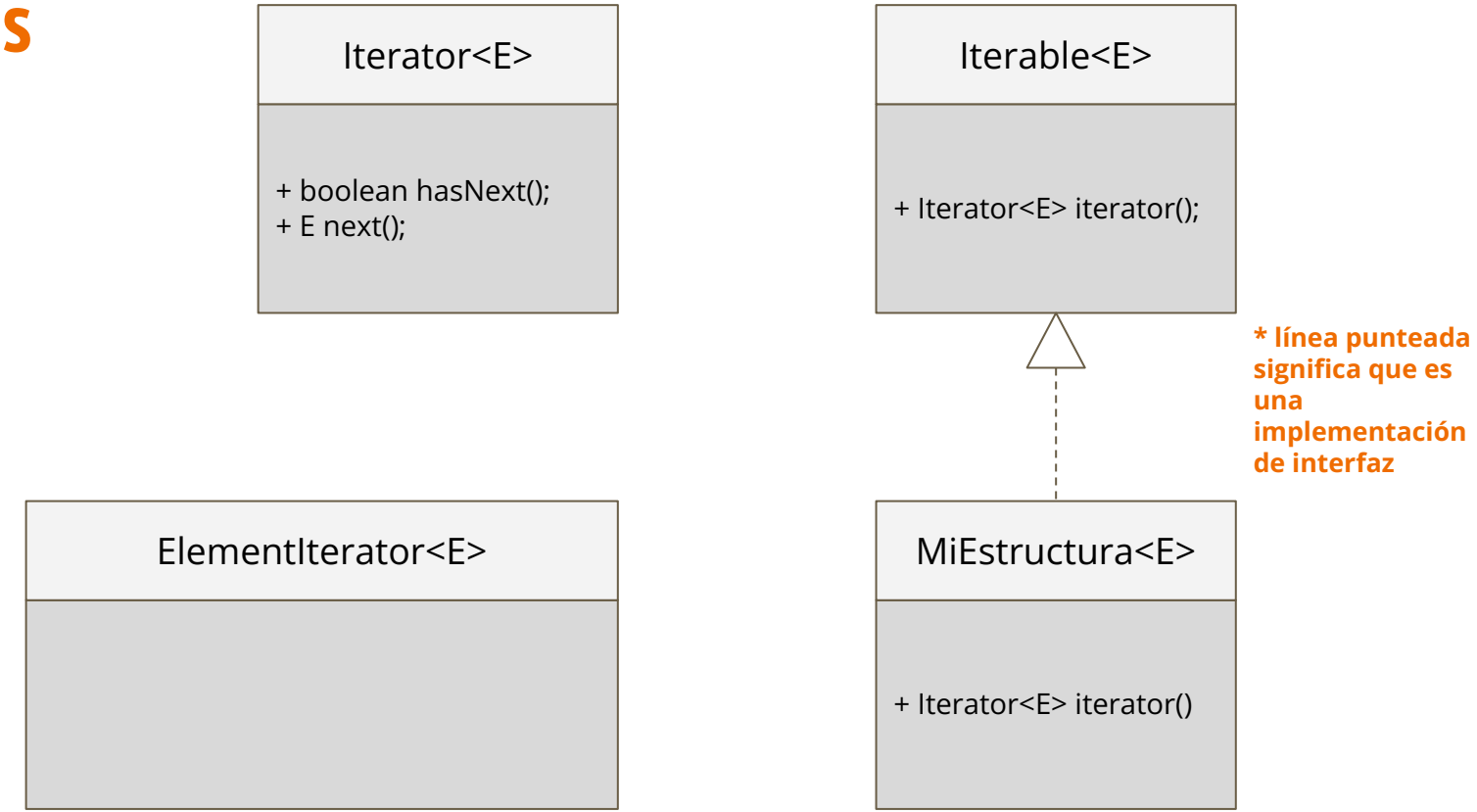


*** línea punteada significa que es una implementación de interfaz**

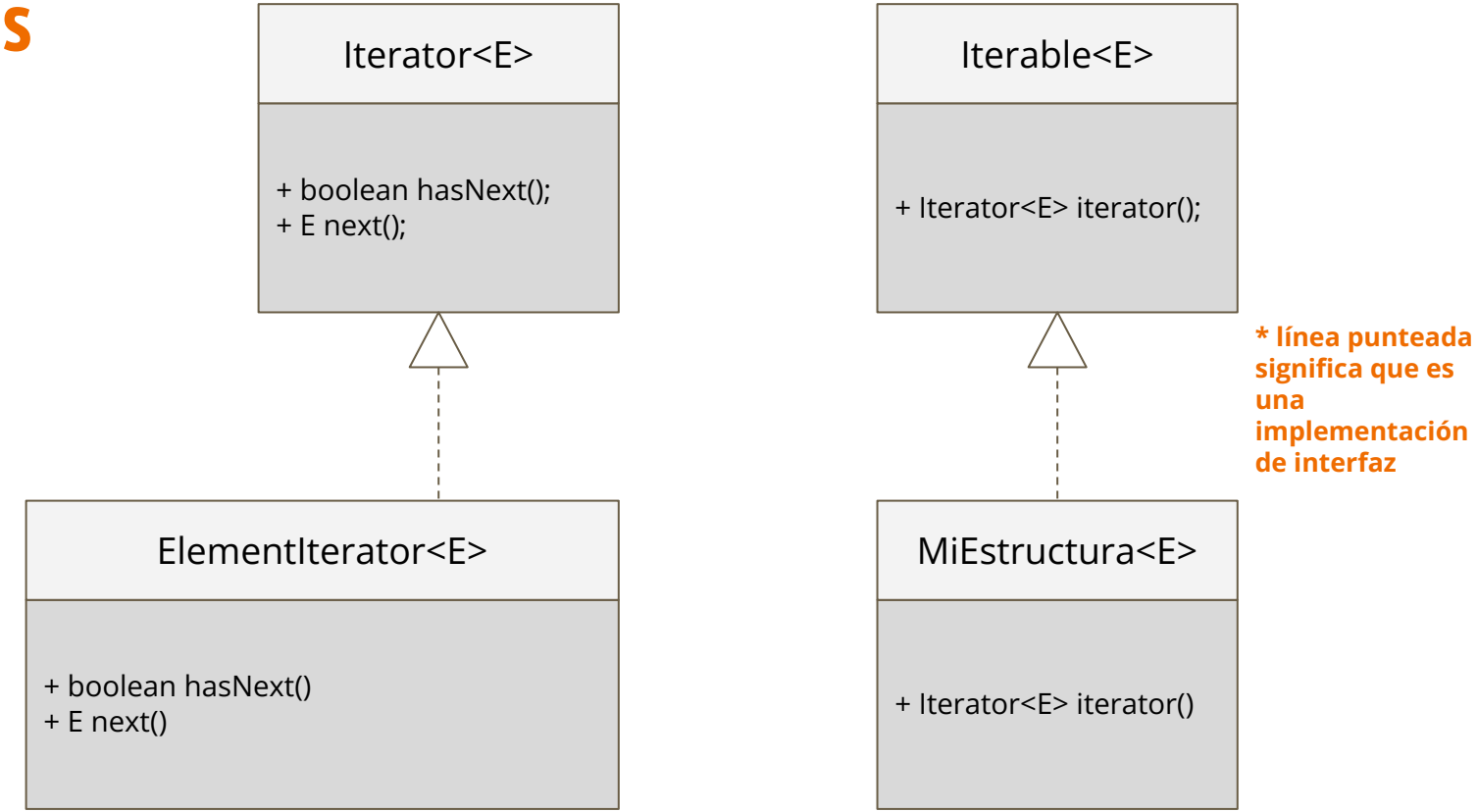
Iteradores


Iterator<E>
+ boolean hasNext();
+ E next();
Iterable<E>
+ Iterator<E> iterator();
* línea punteada significa que es una implementación de interfaz
MiEstructura<E>
+ Iterator<E> iterator()

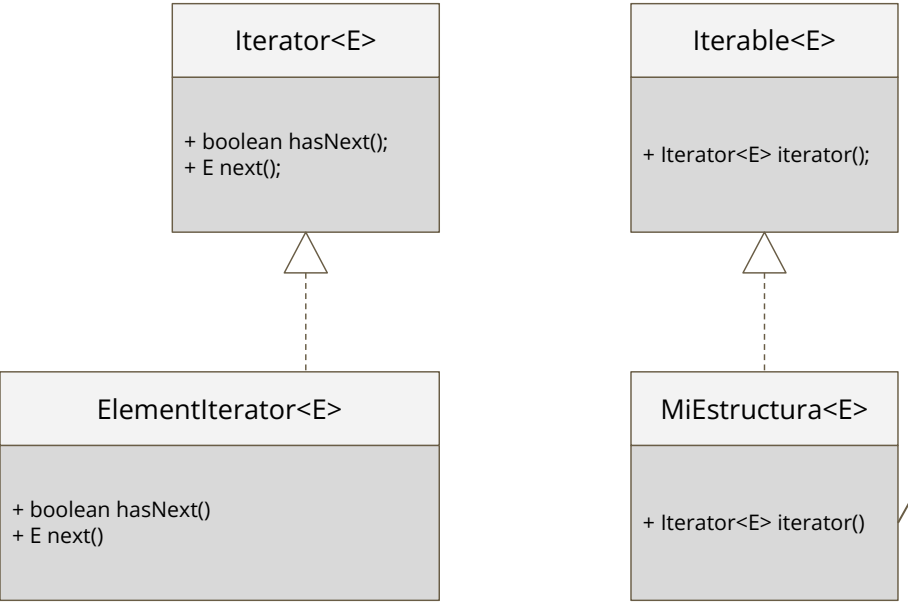
Iteradores


Iterator<E>
+ boolean hasNext();
+ E next();
Iterable<E>
+ Iterator<E> iterator();
* línea punteada significa que es una implementación de interfaz
ElementIterator<E>
MiEstructura<E>
+ Iterator<E> iterator()

Iteradores


Iterator<E>
+ boolean hasNext();
+ E next();
Iterable<E>
+ Iterator<E> iterator();
* línea punteada significa que es una implementación de interfaz
ElementIterator<E>
+ boolean hasNext()
+ E next()
MiEstructura<E>
+ Iterator<E> iterator()

Iteradores



Como la clase ListaDoblementeEnlazada<E> implementa a la interfaz PositionList<E> y esa interfaz extiende a Iterable<E>, **debemos implementar el método iterator en la clase ListaDoblementeEnlazada<E>.**

```
public Iterator<E> iterator() {
    return new ElementIterator<E>(this);
}
```

Iteradores

Para implementar un Iterador tenemos que tener en cuenta que el mismo consiste en:

- Una secuencia
- Un elemento actual
- Un mecanismo para pasar al próximo elemento y hacerlo el elemento actual

Iterador para PositionList: hasNext() y next()

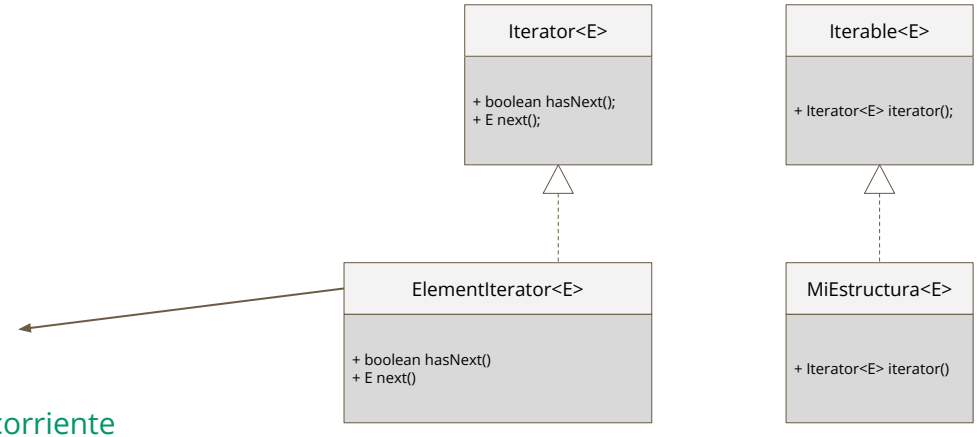
Si la lista está vacía:

- hasNext() → Falso
- next() → Excepción

Si la lista tiene un elemento:

- 1er hasNext() → True
- next() → me da el primer elemento y se posiciona
- 2do hasNext() → False
- next() → Excepción

Iterador para PositionList




```
public class ElementIterator<E> implements Iterator<E> {

	protected PositionList<E> list; // lista a iterar

	protected Position<E> cursor; // Posición del elemento corriente donde está el iterator

	public ElementIterator(PositionList<E> l){

		list = l;

		if (list.isEmpty()) {

			cursor = null;

		} else {

			cursor = list.first();
		}
	}
}
```

Requiere:

```
import java.util.Iterator;
import java.util.NoSuchElementException;

import ar.edu.uns.cs.ed.tdas.Position;
```

Iterador para PositionList

```
/**
* Indica si existe un siguiente elemento, al elemento actual.
* @return true, si existe un elemento siguiente al elemento actual en la lista iterable.
* @return false, si no existe elemento siguiente al elemento actual en la lista iterable.
*/

        public boolean hasNext(){

                return cursor != null;

        }
```

```
/**
* @return el siguiente elemento en la lista iterable.
* @throws NoSuchElementException si no hay elemento siguiente al elemento actual, en la lista iterable.
*/
    public E next() {

        if (cursor == null){

            throw new NoSuchElementException("No existe un siguiente elemento.");

        }

        E toReturn = cursor.element();

        cursor = (cursor == list.last()) ?
                    null : list.next(cursor);

         return toReturn;

}
```

Método Positions

Cómo habrán notado, la interface PositionList<E> tiene un método con la siguiente signatura:

```
public Iterable<Position<E>> positions();
```

Este método es muy útil cuando necesitamos recorrer las posiciones de una estructura (no sus elementos).

Recordemos que según la jerarquía de herencia, un Iterable es cualquier cosa que extienda a la interface Iterable, por lo tanto una PositionList es un iterable.

Método positions

```
public Iterable<Position<E>> positions() {
    PositionList<Position<E>> pl = new ListaDoblementeEnlazada<Position<E>>();
        if(cantElems!=0) {
            DNode<E> n = header.getNext();
            while( n != trailer ) {
                pl.addLast(n);
                n =n.getNext() ;
            }
            }
    return pl;
}
```

Break

Problema

Dada una lista de números enteros, muestre por consola solo sus elementos pares.

¿Qué tipo de recorrido es? → Exhaustivo

Usando el iterador de la lista

```
public void elementosPares(PositionList<Integer> l){
        Iterator<Integer> it= l.iterator();
        while(it.hasNext()){
          Integer n= it.next();
          if(n%2==0)
               System.out.println(n);}
}
```

Usando la lista como un iterable

```
public void elementosPares(PositionList<Integer> l){
   for(Integer n: l)
         if(n%2==0)
               System.out.println(n);}
}
```

Sentencia for each

La sentencia for-each de Java sirve para recorrer estructuras que son Iterables.

El recorrido es exhaustivo, no se puede agregar una condición de corte al for each.

```
for(Integer n: l)
```

Esa sentencia se lee como “para cada entero n sacado de la lista l”, la estructura que esté después de los : debe ser si o si iterable (como nuestras listas).

Problema

Escriba un método que reciba una lista de enteros y retorne la posición del primer elemento par de la lista. Si la lista no tiene elementos pares debe retornar nulo.

¿Qué tipo de recorrido es? → No Exhaustivo

El iterador convencional no me sirve porque no puedo obtener las posiciones, por lo tanto utilizaremos el método positions.

Pero... positions retorna un iterable y necesitamos un recorrido no exhaustivo!



Usando positions

```
public Position<Integer> primerPar(PositionList<Integer> l){
  Position<Integer> toRet=null;
  for(Position<Integer> p: l.positions())
        if(p.element()%2==0){
              toRet=p;
              break;}
   return toRet; }
```

Usando positions

```
public Position<Integer> primerPar(PositionList<Integer> l){
  for(Position<Integer> p: l.positions())
        if(p.element()%2==0)
              return p;
}
```

Ojo con esta alternativa que si nunca entra al if el método queda sin valor de retorno, por lo tanto así como está no compilaría

Usando positions

```
public Position<Integer> primerPar(PositionList<Integer> l){
  Position<Integer> toRet=null; Position<Integer> p=null;
  boolean encontre=false;
  Iterator<Position<Integer>> it= l.positions().iterator();
  while(it.hasNext() && !encontre){
            p=it.next();
            if(p.element()%2==0)
                {toRet=p; encontre=true;}
   return toRet; }
```

THE END